

Data Wrangling 🤠

Conditional Statements

Lesson 6

if Statements

- Runs a block of code only **if** a condition is **True**

```
if [boolean expression]:  
    [code to run if True]
```

- Code inside the block must be **indented**
- If the condition is **False**, the block is skipped entirely

Demo: if-then

```
# weather.py
temp = int(input("What is the temperature today: "))

if temp > 70:
    print("It is warm today!") # DISCUSS IDENTATION

print(f"Enjoy the weather.")
```

Run the program with the command: `python -m lessons.weather`

if-else Statements

- Runs *first* block if `True`, and the *second* block if `False`

```
if [boolean expression]:  
    [runs when True]  
else:  
    [runs when False]
```

Demo: if-else

```
# weather.py
temp = int(input("What is the temperature today: "))

if temp > 70:
    print("It is warm today!")
else:
    print("Bring a sweater.")

print("Enjoy the weather.")
```

Nested `if-else`

- `if` statements are just statements — they can go **inside** other `if` blocks

```
if temp > 70:
    print("It is warm today!")
else:
    if temp < 40:
        print("Bring a coat.")
    else:
        print("Bring a sweater.")

print("Enjoy the weather.")
```

Logical Operators

- Sometimes one condition isn't enough
- Logical operators let us combine boolean expressions

Operator	Meaning	Example
<code>and</code>	Both must be <code>True</code>	<code>x > 0 and x < 10</code>
<code>or</code>	At least one must be <code>True</code>	<code>x == 0 or x == 1</code>
<code>not</code>	Flips <code>True</code> to <code>False</code>	<code>not x == 0</code>

Demo: Compound Conditions

```
# weather.py
temperature = int(input("What is the temperature today: "))
cloudy = bool(input("Is it cloudy today: "))
describe = ""

# if cloudy is yes ...
# if cloudy is no ....

if temperature > 70 and not cloudy:
    describe = "sunny"
else:
    if temperature > 70:
        describe = "warm"
    else:
        describe = "cool"

print(f"Enjoy the {describe} weather.")
```

elif Statements

- Checks an additional condition when the previous `if` was `False`
- You can chain as many `elif` blocks as needed
- Only the first matching condition runs; the rest are skipped

```
if [boolean expression]:  
    [runs when True and skip rest]  
elif [boolean expression]:  
    [runs when True and skip rest]  
else:  
    [runs when everything else False]
```

Demo: elif

```
# weather.py
temp = int(input("What is the temperature today: "))
cloudy = bool(input("Is it cloudy today: "))
describe = ""

if temp > 70 and cloudy == False:
    describe = "sunny"
elif temp > 70:
    describe = "warm"
elif temp < 40:
    describe = "cold"
else:
    describe = "cool"

print(f"Enjoy the {describe} weather.")
```